

C Programming Debugging assignment

Name- Aaryan Pratap Singh

Sap ID- 590032842

Batch - 15

Course- B. Tech

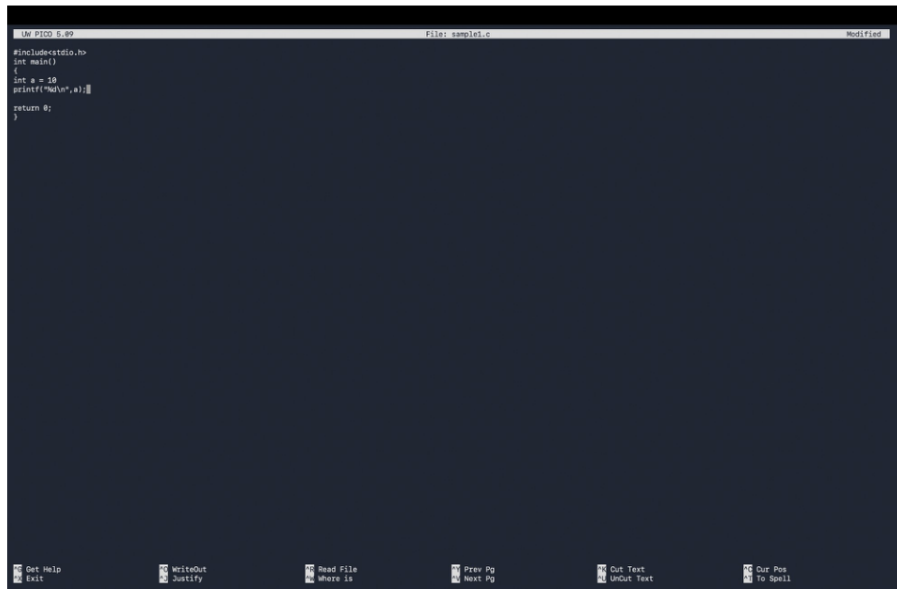
1. Fundamentals of Errors and Debugging

An error is a mistake in source code. A bug is a defect that causes incorrect or unexpected behavior. Debugging is the systematic process of finding, analyzing, and removing such defects.

Error Type	Meaning	Typical Example
Syntax	Violates C grammar	Missing semicolon
Compilation/Type	Invalid type or declaration	Incompatible assignment
Linker	Required definition cannot be linked	Undefined function
Runtime	Fails during execution	Division by zero / invalid memory
Logical	Runs but gives wrong result	Incorrect loop condition

Case 1: Syntax & Compilation Error

A missing semicolon prevents successful compilation. Clang identifies the source line and reports that a semicolon is expected.

The image shows a code editor window titled "File: sample.c" with a dark background. The code is as follows:

```
#include <stdio.h>
int main()
{
    int a = 10;
    printf("Value: %d", a);
    return 0;
}
```

The code is syntactically correct in the snippet shown, but the error message in the terminal below indicates a missing semicolon. The terminal window shows the output of a compilation command, which includes the error message: "error: ';' expected". The terminal also shows the output of the program, which is "Value: 10". The code editor has a menu bar at the top with "File", "Edit", "Format", "Tools", and "Help". The terminal window has a title bar that says "Terminal" and a menu bar with "File", "Edit", "Format", "Tools", and "Help".

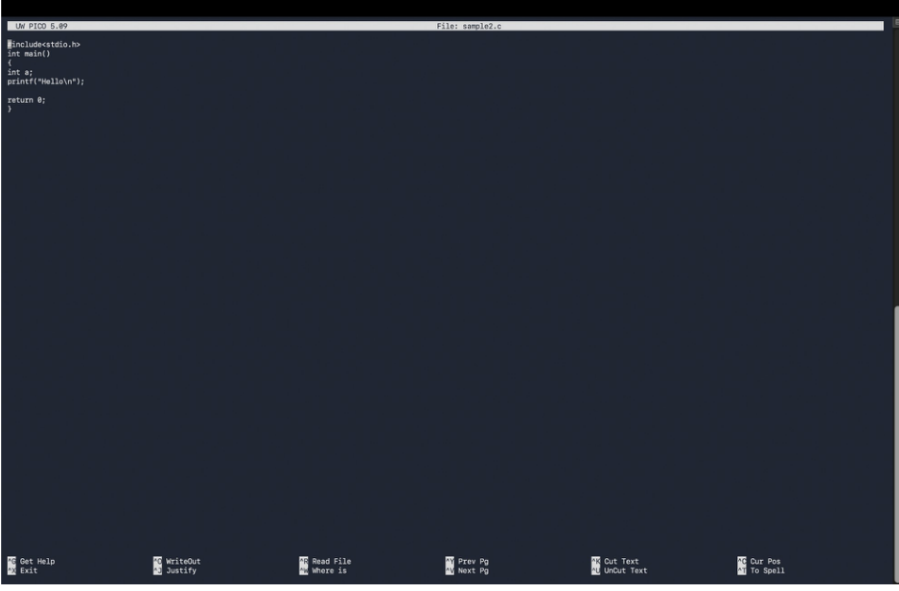
Source code containing the syntax error.

```
ohruvandy@MYS-MacBook-Air: ~ % clang sample1.c -o sample1.o
sample1.c:11:11: error: expected ';' at end of declaration
4 | int a = 10;
  |           ^
1 error generated.
ohruvandy@MYS-MacBook-Air: ~ %
```

Terminal output showing the Clang syntax-error diagnostic.

Case 2: Compiler Warnings (-Wall, -Wextra)

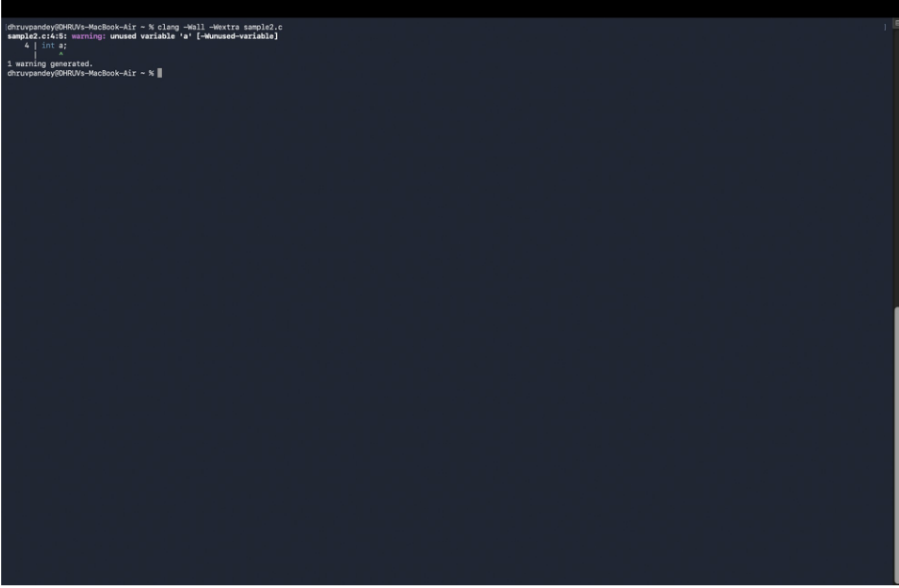
Warnings identify suspicious code that is still legal C. Here, variable a is declared but never used.



```
UW PICO 5.89 File: sample2.c
#include <stdio.h>
int main()
{
    int a;
    printf("hello\n");
    return 0;
}
```

Get Help WriteOut Read File Prev Pg Cut Text Our Pos
Exit Justify Where Is Next Pg UnCut Text To Spell

Program containing an unused variable.



```
chrispandey@HRVys-MacBook-Air: ~ % clang -Wall -Wextra sample2.c
sample2.c:4:5: warning: unused variable 'a' [-Wunused-variable]
    a | int a;
      +
1 warning generated.
chrispandey@HRVys-MacBook-Air: ~ %
```

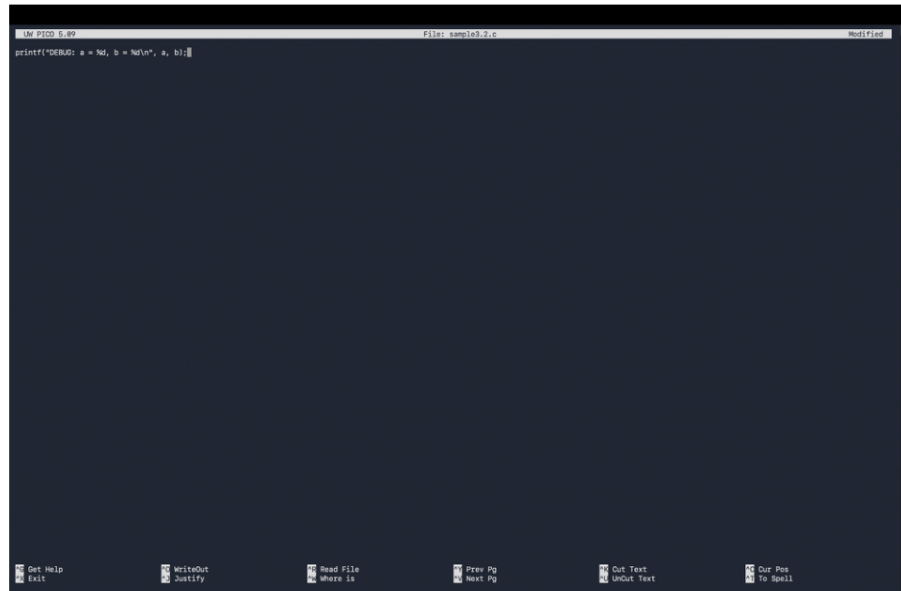
Terminal warning generated with -Wall and -Wextra.

Case 3: printf-Based Debugging (Logical Bug)

Debug print statements expose the values of `i` and `sum` on each loop iteration. The trace shows the program reaches a final sum of 10.

```
dhruvpandey@HRV-MacBook-Air: ~ % clang sample3.c -o sample3
dhruvpandey@HRV-MacBook-Air: ~ % ./sample3
DEBUG: sum = 0
DEBUG: i = 0
DEBUG: sum = 1
DEBUG: i = 1
DEBUG: sum = 3
DEBUG: i = 2
DEBUG: sum = 6
DEBUG: i = 3
DEBUG: sum = 10
dhruvpandey@HRV-MacBook-Air: ~ %
```

Terminal trace produced by printf-based debugging.



The image shows a GDB terminal window with a dark background. The title bar at the top reads "UW PIC0 6.89" on the left, "File: sample3.2.c" in the center, and "Modified" on the right. The main area of the window contains a single line of text: `printf("DEBUG: a = %d, b = %d\n", a, b);`. At the bottom of the window, there is a row of icons and labels for various GDB commands: "Get Help", "Exit", "WriteOut", "Justify", "Read File", "Where Is", "Prev Pg", "Next Pg", "Cut Text", "UnCut Text", "Our Pos", and "To Spell".

Example of a debug printf statement.

Case 4: Debugging a Logical Error using GDB

The loop uses $i < 5$, so 5 is never included. GDB can stop on the sum statement and display i and sum at each iteration. The actual result is 10; the intended result is 15.

The image consists of two screenshots of a GDB online debugger interface. The top screenshot shows the program starting at line 7, where `sum = sum + i;` is executed, resulting in `sum = 1`. The bottom screenshot shows the program continuing to line 7, where `sum = sum + i;` is executed again, resulting in `sum = 3`. The program continues to execute, and the final output is `Sum = 10`.

```
1 #include <stdio.h> // Line 1
2 int main() // Line 2
3 { // Line 3
4     int i, sum = 0; // Line 4
5     for(i = 1; i < 5; i++) // Line 5 (Logical Bug: should be i <= 5)
6     { // Line 6
7         sum = sum + i; // Line 7
8     } // Line 8
9     printf("Sum = %d\n", sum); // Line 9
10    return 0; // Line 10
11 }
```

Debug Console

Breakpoint 1, main () at main.c:7
7: sum = sum + i; // Line 7
(gdb) print i
i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.
Breakpoint 1, main () at main.c:7
7: sum = sum + i; // Line 7
i: i = 2
2: sum = 1
(gdb) continue
Continuing.
Breakpoint 1, main () at main.c:7
7: sum = sum + i; // Line 7
i: i = 3
2: sum = 3
(gdb) continue
Continuing.
Breakpoint 1, main () at main.c:7
7: sum = sum + i; // Line 7
i: i = 4
2: sum = 6
(gdb) continue
Continuing.
Sum = 10

GDB observation: when `i` reaches 5, the loop condition becomes false before the body executes. Therefore 5 is omitted from the sum.

```

1 #include <stdio.h> // Line 1
2 int main() // Line 2
3 { // Line 3
4     int i, sum = 0; // Line 4
5     for(i = 1; i < 5; i++) // Line 5 (Logical Bug: should be i <= 5)
6     { // Line 6
7         sum = sum + i; // Line 7
8     } // Line 8
9     printf("Sum = %d\n", sum); // Line 9
10    return 0; // Line 10
11 }

```

```

(gdb) break?
Undefined command: "break?". Try "help".
(gdb) break
No default breakpoint address now.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".
Sum = 10
[Inferior 1 (process 3007) exited normally]
(gdb) print i
Undefined command: "printi". Try "help".
(gdb) print i
No symbol "i" in current context.
(gdb)

```

OnlineGDB logical-error debugging session.

Case 5: Debugging Functions (next vs. step)

GDB next executes a function call without entering it, while step enters the called function so its statements and local values can be inspected.

```

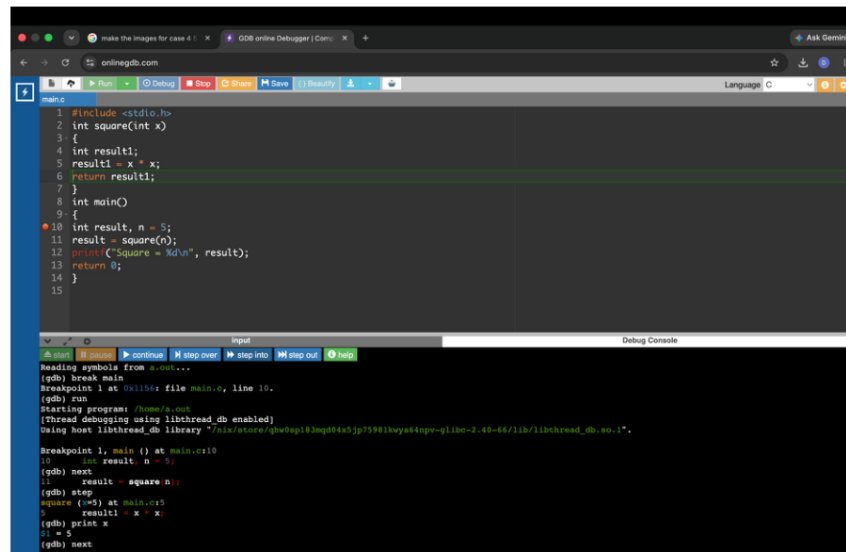
1 #include <stdio.h>
2 int square(int x)
3 {
4     int result;
5     result = x * x;
6     return result;
7 }
8 int main()
9 {
10    int result, n = 5;
11    result = square(n);
12    printf("Square = %d\n", result);
13    return 0;
14 }

```

```

Reading symbols from a.out...
(gdb) break main
Breakpoint 1 at 0x1154: file main.c, line 10.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".
Breakpoint 1, main () at main.c:10
10     int result, n = 5;
(gdb) next
11     result = square(n);
(gdb) next
12     printf("Square = %d\n", result);
(gdb) print result
$1 = 25
(gdb)

```



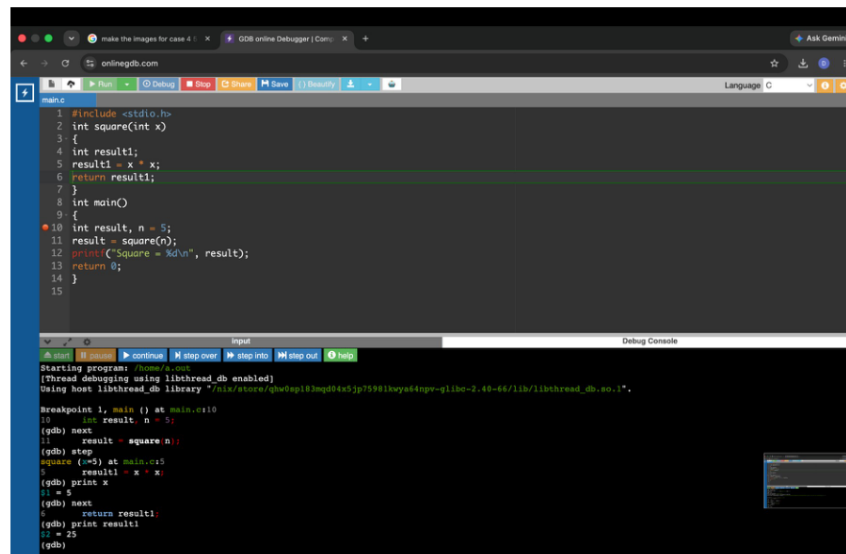
```

1 #include <stdio.h>
2 int square(int x)
3 {
4     int result1;
5     result1 = x * x;
6     return result1;
7 }
8 int main()
9 {
10     int result, n = 5;
11     result = square(0);
12     printf("Square = %d\n", result);
13     return 0;
14 }
15

(gdb) break main
Breakpoint 1 at 0x1154: file main.c, line 10.
(gdb) run
Starting program: /home/a.out
(Thread debugging using libthread_db enabled)
Using host libthread_db library "/lib64/libthread_db.so.1".

Breakpoint 1, main () at main.c:10
10     int result, n = 5;
(gdb) next
11     result = square(0);
(gdb) step
square (=5) at main.c:5
5     result1 = x * x;
(gdb) print x
x = 5
(gdb) next

```



```

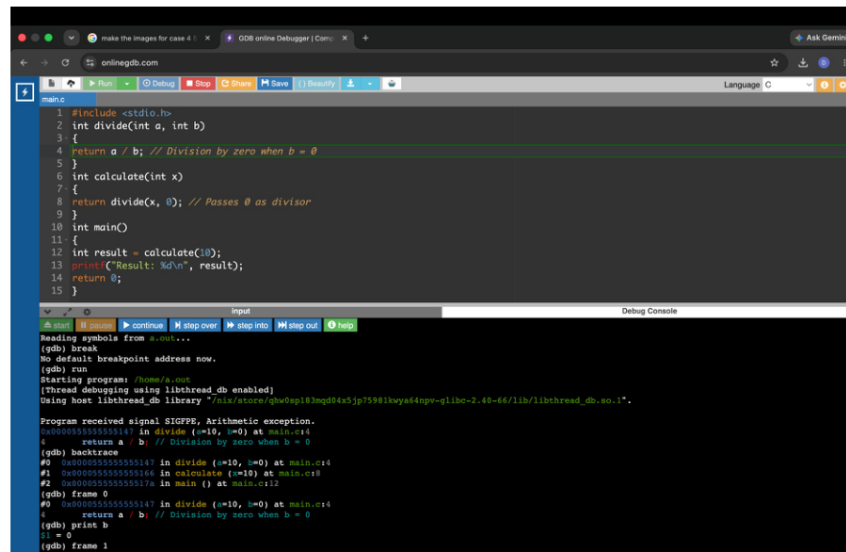
Starting program: /home/a.out
(Thread debugging using libthread_db enabled)
Using host libthread_db library "/lib64/libthread_db.so.1".

Breakpoint 1, main () at main.c:10
10     int result, n = 5;
(gdb) next
11     result = square(n);
(gdb) step
square (=5) at main.c:5
5     result1 = x * x;
(gdb) print x
x = 5
(gdb) next
6     return result1;
(gdb) print result1
result1 = 25
(gdb)

```

Case 6: Runtime Error using GDB Backtrace

A division by zero produces SIGFPE. The backtrace command shows the chain divide -> calculate -> main, making it possible to locate the origin of the crash.



```

1 #include <stdio.h>
2 int divide(int a, int b)
3 {
4     return a / b; // Division by zero when b = 0
5 }
6 int calculate(int x)
7 {
8     return divide(x, 0); // Passes 0 as divisor
9 }
10 int main()
11 {
12     int result = calculate(10);
13     printf("Result: %d\n", result);
14     return 0;
15 }

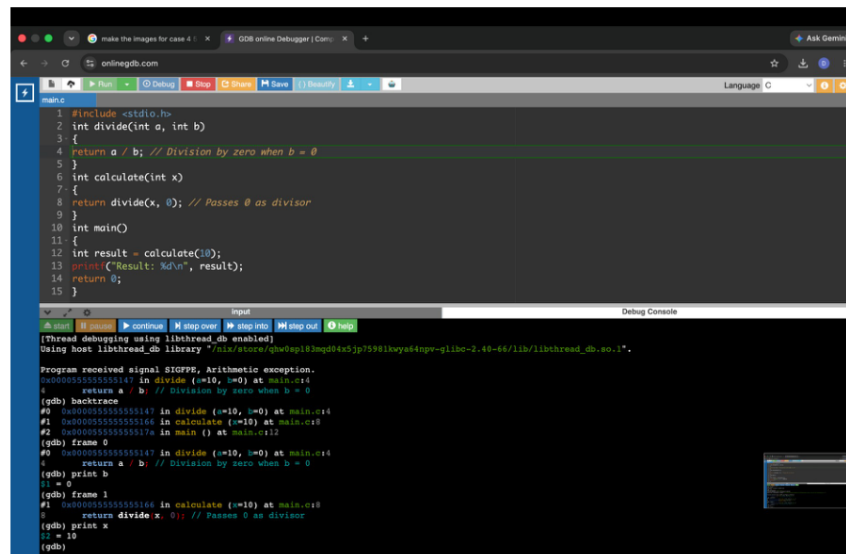
```

Debug Console

```

(gdb) break
No default breakpoint address now.
(gdb) run
Starting program: /home/a.out
(Thread debugging using libthread_db enabled)
Using host libthread_db library "/lib64/libthread_db.so.1".
Program received signal SIGFPE, Arithmetic exception.
0x0000555555551147 in divide (a=10, b=0) at main.c:4
4     return a / b; // Division by zero when b = 0
(gdb) backtrace
#0 0x0000555555551147 in divide (a=10, b=0) at main.c:4
#1 0x0000555555551146 in calculate (x=10) at main.c:8
#2 0x000055555555117a in main () at main.c:12
(gdb) frame 0
#0 0x0000555555551147 in divide (a=10, b=0) at main.c:4
4     return a / b; // Division by zero when b = 0
(gdb) print b
b = 0
(gdb) frame 1

```



```

1 #include <stdio.h>
2 int divide(int a, int b)
3 {
4     return a / b; // Division by zero when b = 0
5 }
6 int calculate(int x)
7 {
8     return divide(x, 0); // Passes 0 as divisor
9 }
10 int main()
11 {
12     int result = calculate(10);
13     printf("Result: %d\n", result);
14     return 0;
15 }

```

Debug Console

```

(gdb) break
Using host libthread_db library "/lib64/libthread_db.so.1".
Program received signal SIGFPE, Arithmetic exception.
0x0000555555551147 in divide (a=10, b=0) at main.c:4
4     return a / b; // Division by zero when b = 0
(gdb) backtrace
#0 0x0000555555551147 in divide (a=10, b=0) at main.c:4
#1 0x0000555555551146 in calculate (x=10) at main.c:8
#2 0x000055555555117a in main () at main.c:12
(gdb) frame 0
#0 0x0000555555551147 in divide (a=10, b=0) at main.c:4
4     return a / b; // Division by zero when b = 0
(gdb) print b
b = 0
(gdb) frame 1
#1 0x0000555555551146 in calculate (x=10) at main.c:8
8     return divide(x, 0); // Passes 0 as divisor
(gdb) print x
x = 10
(gdb)

```

Case 7: Array Out-of-Bounds Error using GDB

An array of five integers has valid indices 0 through 4. The condition `i <= 5` also accesses `a[5]`, which is outside the array and causes undefined behavior.

```

1 #include <stdio.h>
2 int main()
3 {
4     int a[5];
5     int i;
6     for(i = 0; i <= 5; i++)
7     {
8         scanf("%d", &a[i]); //line 8
9     }
10    return 0;
11 }

```

Debug Console

```

(gdb) break 8
Breakpoint 1 at 0x114: file main.c, line 8.
(gdb) run
Starting program: /home/a.out
(Thread debugging using libthread_db enabled)
Using host libthread_db library "/nix/store/gw8qg18mgdtt5jp75981bvy64npv-glibc-2.40-66/lib/libthread_db.so.1".
Breakpoint 1, main () at main.c:8
> scanf("%d", &a[i]); //line 8
(gdb) print &a[0]
Undefined command: "print". Try "help".
(gdb) print &a[0]
$1 = (int *) 0x00000000
(gdb) print &a[4]
$2 = (int *) 0x00000000
(gdb) print &a[5]
$3 = (int *) 0x00000000
(gdb)

```

Case 8: Segmentation Fault (SIGSEGV) using GDB

The pointer `p` is NULL (address 0x0). Writing through it with `*p = 10` attempts to access invalid memory, so the program receives SIGSEGV.

```

1 #include <stdio.h>
2 int main()
3 {
4     int *p = NULL; // Pointer points to address 0x0
5     *p = 10; // Attempting to write to address 0x0 causes Segmentation Fault
6     printf("%d\n", *p);
7     return 0;
8 }

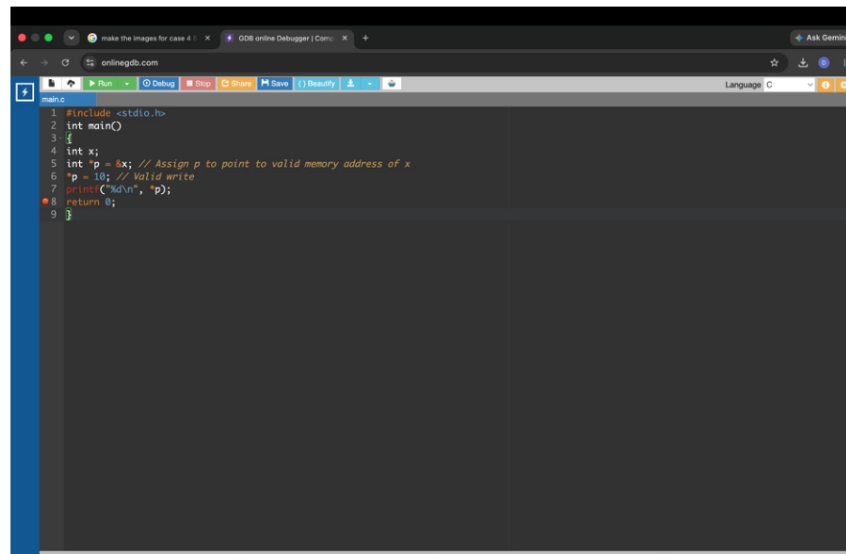
```

Debug Console

```

(gdb) break 6
Breakpoint 1 at 0x0000000000001174 in main at main.c:6
(gdb) run
Starting program: /home/a.out
(Thread debugging using libthread_db enabled)
Using host libthread_db library "/nix/store/gw8qg18mgdtt5jp75981bvy64npv-glibc-2.40-66/lib/libthread_db.so.1".
Program received signal SIGSEGV, Segmentation fault.
0x0000000000001174 in main () at main.c:6
> *p = 10; // Attempting to write to address 0x0 causes Segmentation Fault
(gdb) print p
$1 = (int *) 0x0
(gdb) print *p
Cannot access memory at address 0x0
(gdb)

```



The screenshot shows a web browser window with the URL 'onlinegdb.com'. The page title is 'GDB online Debugger | Cam...'. The browser's address bar shows 'onlinegdb.com'. The main content area displays a C program in a dark-themed editor. The code is as follows:

```
1 #include <stdio.h>
2 int main()
3 {
4     int x;
5     int *p = &x; // Assign p to point to valid memory address of x
6     p = 10; // Valid write
7     printf("%d\n", *p);
8     return 0;
9 }
```

The code is syntactically correct, but it contains a logical error: on line 6, the pointer `p` is assigned the value 10, which is not a valid memory address. This will cause a segmentation fault when the program reaches line 7 and attempts to dereference `*p`. The GDB interface includes standard controls like 'Run', 'Debug', 'Stop', 'Share', 'Save', and 'Security' buttons at the top. A 'Language: C' dropdown is visible on the right side of the editor.

Systematic Debugging Strategy

1. Observe expected vs. actual output
2. Reproduce the issue consistently
3. Compile with `-Wall -Wextra -g`
4. Isolate suspicious code
5. Inspect state using `printf` or GDB
6. Apply one minimal fix
7. Recompile and retest
8. Verify edge cases and boundary inputs

Conclusion

Compiler diagnostics, warning flags, `printf` tracing, and GDB provide complementary ways to locate errors. Breakpoints and variable inspection help with logical bugs; `next` and `step` help analyze functions; `backtrace` identifies crash call chains; and pointer/array inspection helps identify invalid memory access.